

Building Blocks of CNNs

Parham Kebria
Senior Research Scientist
parhamkebria@ieee.org

Advanced Study Institute: Artificial Intelligence for Disaster Management
Orlando, November 17 – 25, 2025

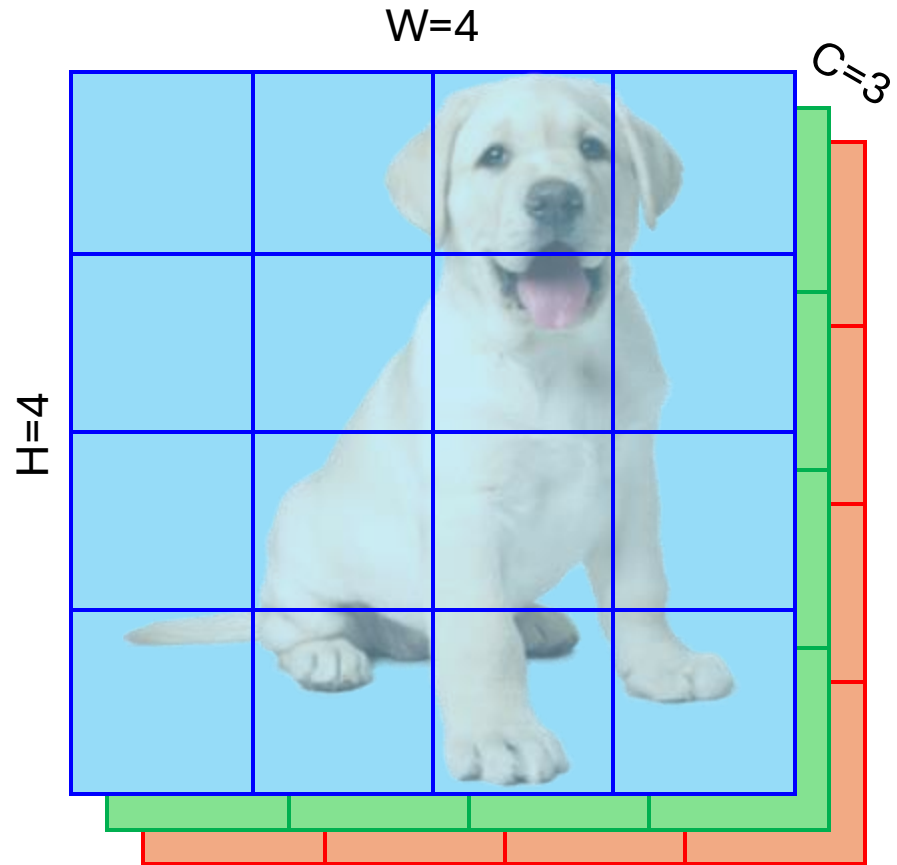


*This activity
is supported by:*

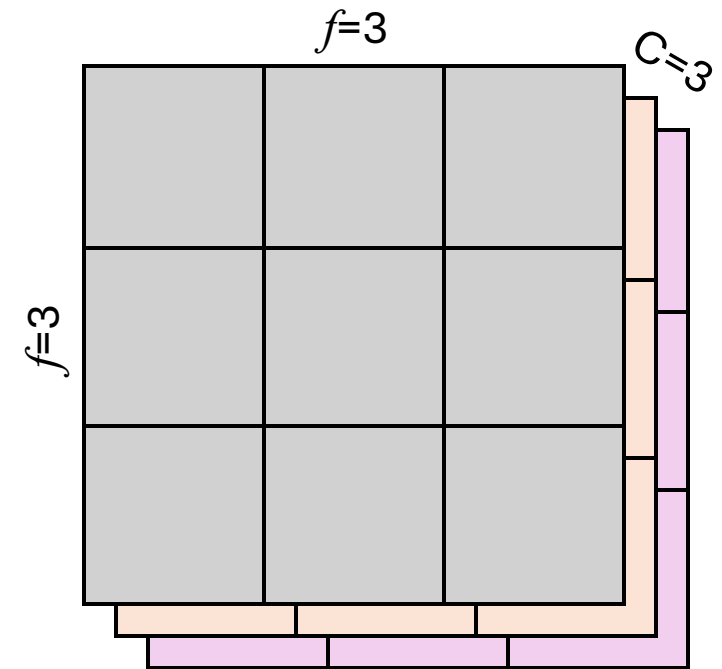
The NATO Science for Peace
and Security Programme

Convolutional Neural Networks (CNN)

The convolution operation



Input Tensor



Convolution filter

Convolutional Neural Networks (CNN)

The convolution operation

i_{11}	i_{12}	i_{13}	i_{14}
i_{21}	i_{22}	i_{23}	i_{24}
i_{31}	i_{32}	i_{33}	i_{34}
i_{41}	i_{42}	i_{43}	i_{44}

Input Tensor

f_{11}	f_{12}	f_{13}
f_{21}	f_{22}	f_{23}
f_{31}	f_{32}	f_{33}

Convolution filter



Convolutional Neural Networks (CNN)

The convolution operation

i_{11} f_{11}	i_{12} f_{12}	i_{13} f_{13}	i_{14}
i_{21} f_{21}	i_{22} f_{22}	i_{23} f_{23}	i_{24}
i_{31} f_{31}	i_{32} f_{32}	i_{33} f_{33}	i_{34}
i_{41}	i_{42}	i_{43}	i_{44}

Input Tensor

$$\begin{aligned} c_{11} = & i_{11} \times f_{11} + i_{12} \times f_{12} + i_{13} \times f_{13} \\ & + i_{21} \times f_{21} + i_{22} \times f_{22} + i_{23} \times f_{23} \\ & + i_{31} \times f_{31} + i_{32} \times f_{32} + i_{33} \times f_{33} \end{aligned}$$

c_{11}

Convolutional Neural Networks (CNN)

The convolution operation

i_{11}	i_{12} f_{11}	i_{13} f_{12}	i_{14} f_{13}
i_{21}	i_{22} f_{21}	i_{23} f_{22}	i_{24} f_{23}
i_{31}	i_{32} f_{31}	i_{33} f_{32}	i_{34} f_{33}
i_{41}	i_{42}	i_{43}	i_{44}

Input Tensor

$$\begin{aligned} c_{12} = & i_{12} \times f_{11} + i_{13} \times f_{12} + i_{14} \times f_{13} \\ & + i_{22} \times f_{21} + i_{23} \times f_{22} + i_{24} \times f_{23} \\ & + i_{32} \times f_{31} + i_{33} \times f_{32} + i_{34} \times f_{33} \end{aligned}$$

c_{11}	c_{12}
----------	----------

Convolutional Neural Networks (CNN)

The convolution operation

i_{11}	i_{12}	i_{13}	i_{14}
i_{21} f_{11}	i_{22} f_{12}	i_{23} f_{13}	i_{24}
i_{31} f_{21}	i_{32} f_{22}	i_{33} f_{23}	i_{34}
i_{41} f_{31}	i_{42} f_{32}	i_{43} f_{33}	i_{44}

Input Tensor

$$\begin{aligned} c_{21} = & i_{21} \times f_{11} + i_{22} \times f_{12} + i_{23} \times f_{13} \\ & + i_{31} \times f_{21} + i_{32} \times f_{22} + i_{33} \times f_{23} \\ & + i_{41} \times f_{31} + i_{42} \times f_{32} + i_{43} \times f_{33} \end{aligned}$$

c_{11}	c_{12}
c_{21}	

Convolutional Neural Networks (CNN)

The convolution operation

i_{11}	i_{12}	i_{13}	i_{14}
i_{21}	i_{22} f_{11}	i_{23} f_{12}	i_{24} f_{13}
i_{31}	i_{32} f_{21}	i_{33} f_{22}	i_{34} f_{23}
i_{41}	i_{42} f_{31}	i_{43} f_{32}	i_{44} f_{33}

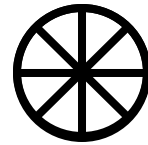
Input Tensor

$$\begin{aligned} c_{22} = & i_{22} \times f_{11} + i_{23} \times f_{12} + i_{24} \times f_{13} \\ & + i_{32} \times f_{21} + i_{33} \times f_{22} + i_{34} \times f_{23} \\ & + i_{42} \times f_{31} + i_{43} \times f_{32} + i_{44} \times f_{33} \end{aligned}$$

c_{11}	c_{12}
c_{21}	c_{22}

i_{11}	i_{12}	i_{13}	i_{14}
i_{21}	i_{22}	i_{23}	i_{24}
i_{31}	i_{32}	i_{33}	i_{34}
i_{41}	i_{42}	i_{43}	i_{44}

Input Tensor



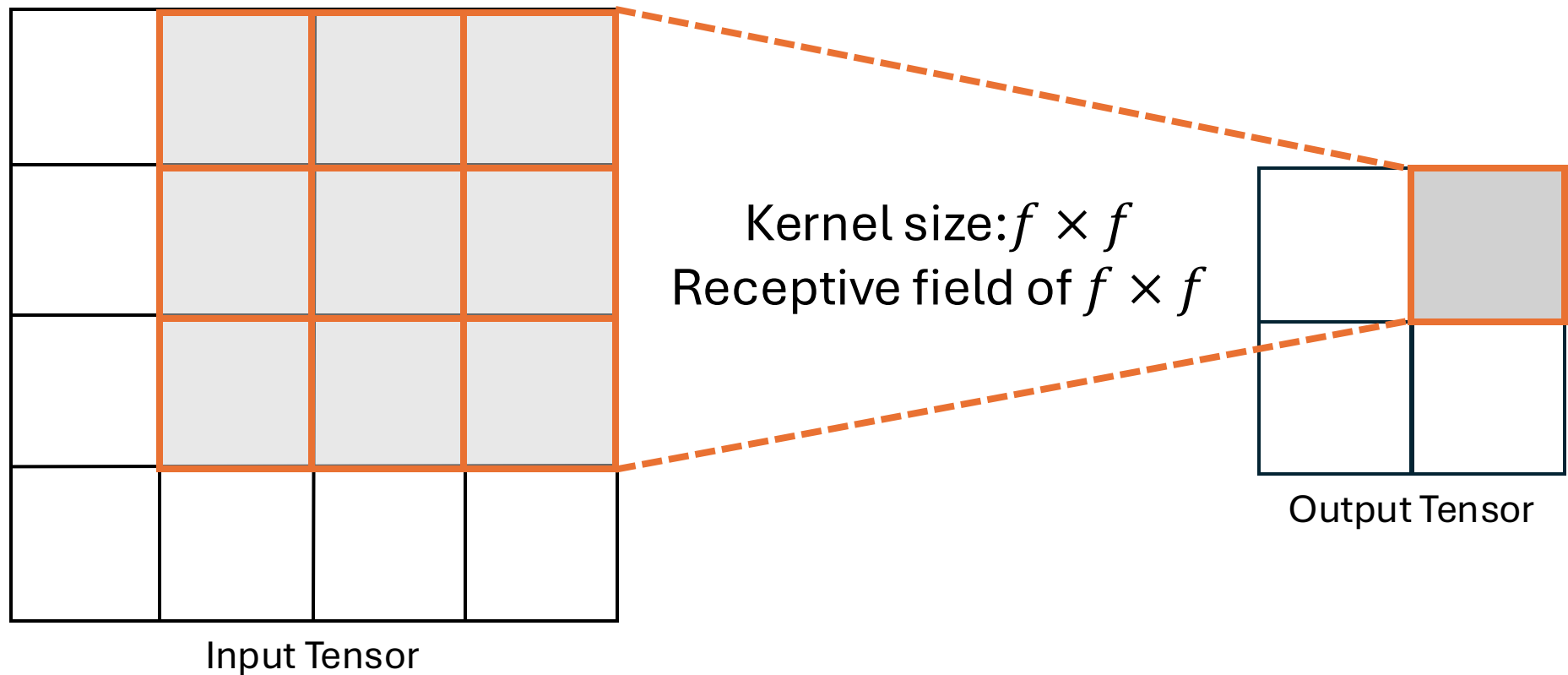
f_{11}	f_{12}	f_{13}
f_{21}	f_{22}	f_{23}
f_{31}	f_{32}	f_{33}

$\Rightarrow$

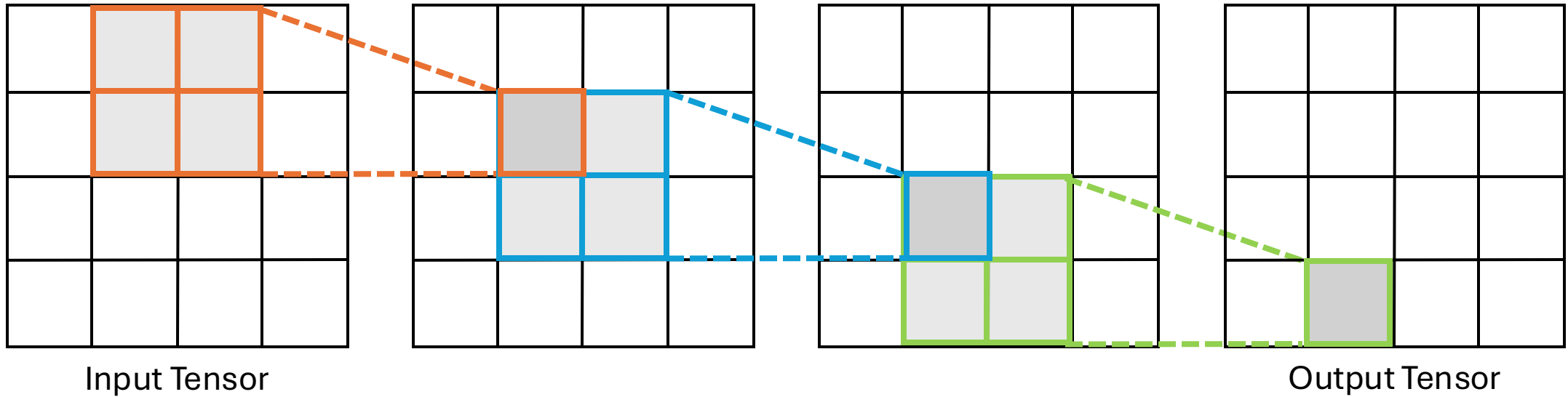
c_{11}	c_{12}
c_{21}	c_{22}

Output Tensor

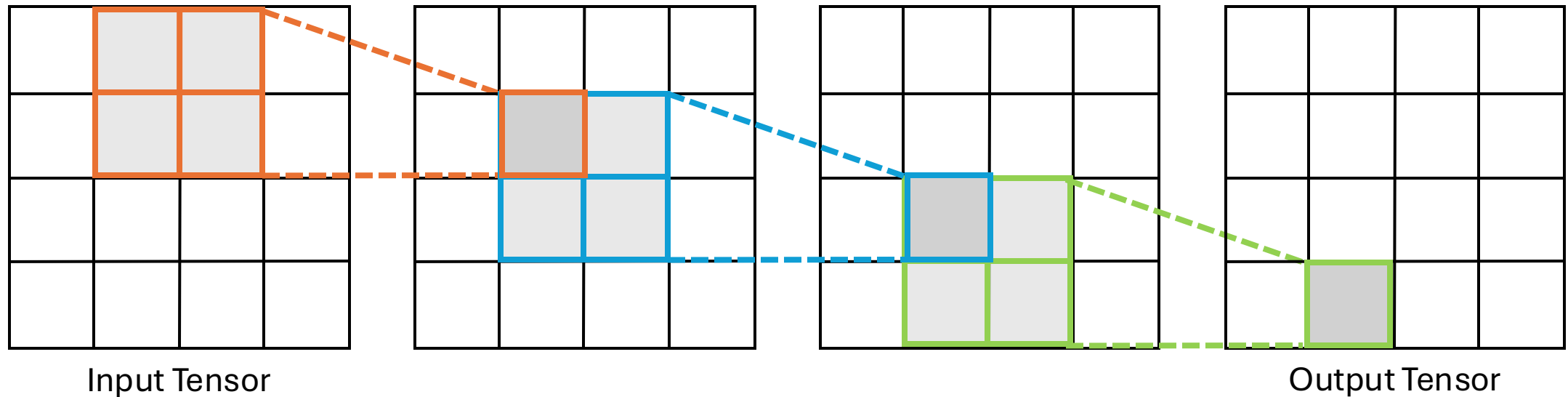
$$c_{pq} = \sum_{i=1}^f \sum_{j=1}^f w_{ij} x_{i+p, j+q}$$



Each conv layer $f \times f$ adds $f - 1$ to the overall receptive field
 $\Rightarrow L$ conv layers results in $1 + L \times (f - 1)$ receptive field

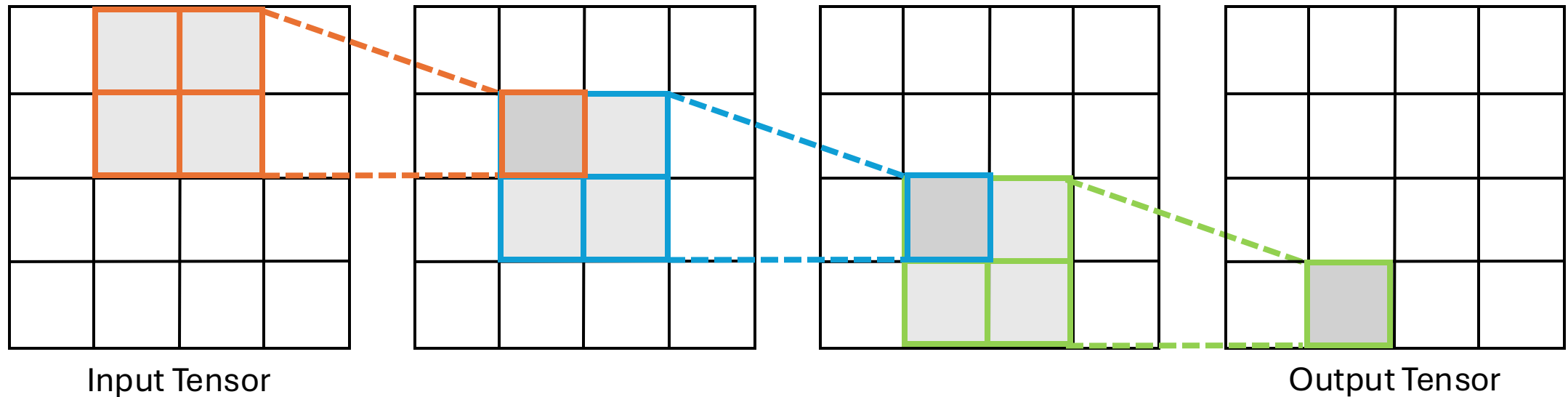


Each conv layer $f \times f$ adds $f - 1$ to the overall receptive field
 $\Rightarrow L$ conv layers results in $1 + L \times (f - 1)$ receptive field



Challenge: For large images we need *MANY* layers
to be able to **see** the whole input image in the output !

Each conv layer $f \times f$ adds $f - 1$ to the overall receptive field
 $\Rightarrow L$ conv layers results in $1 + L \times (f - 1)$ receptive field

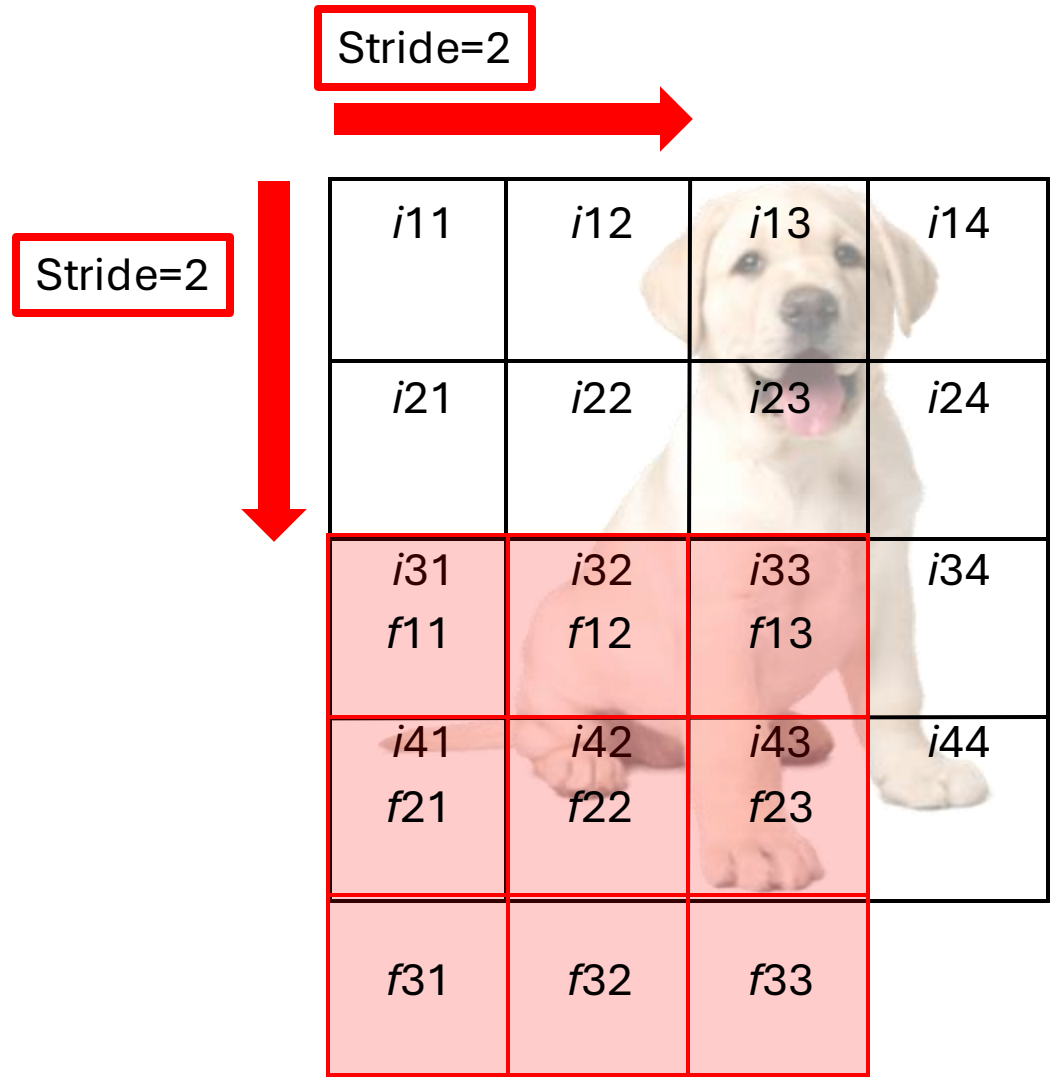
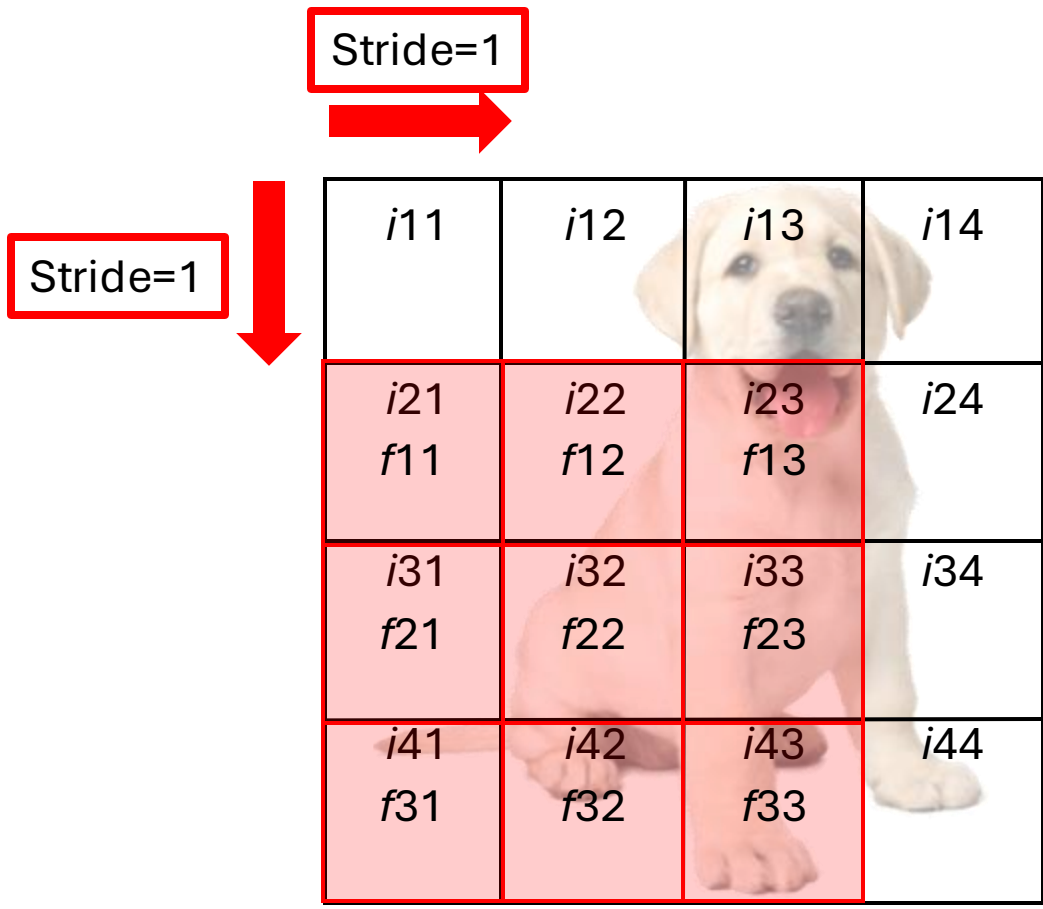


Challenge: For large images we need *MANY* layers
to be able to **see** the whole input image in the output !

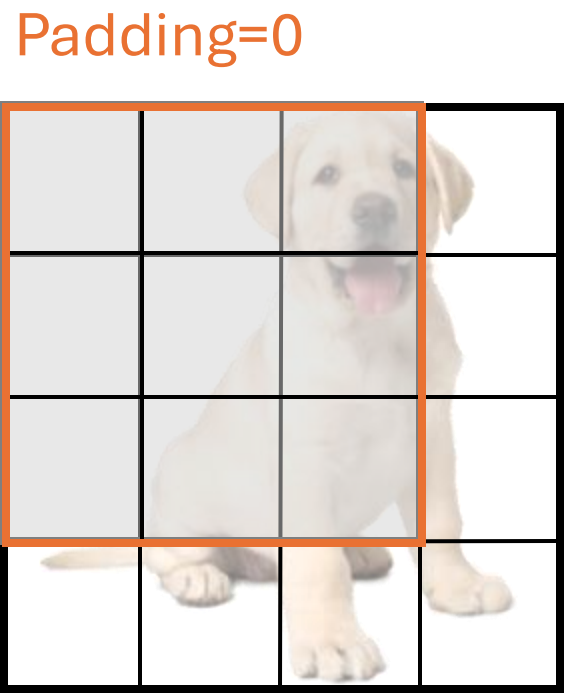
Striding to downsample

Convolutional Neural Networks (CNN)

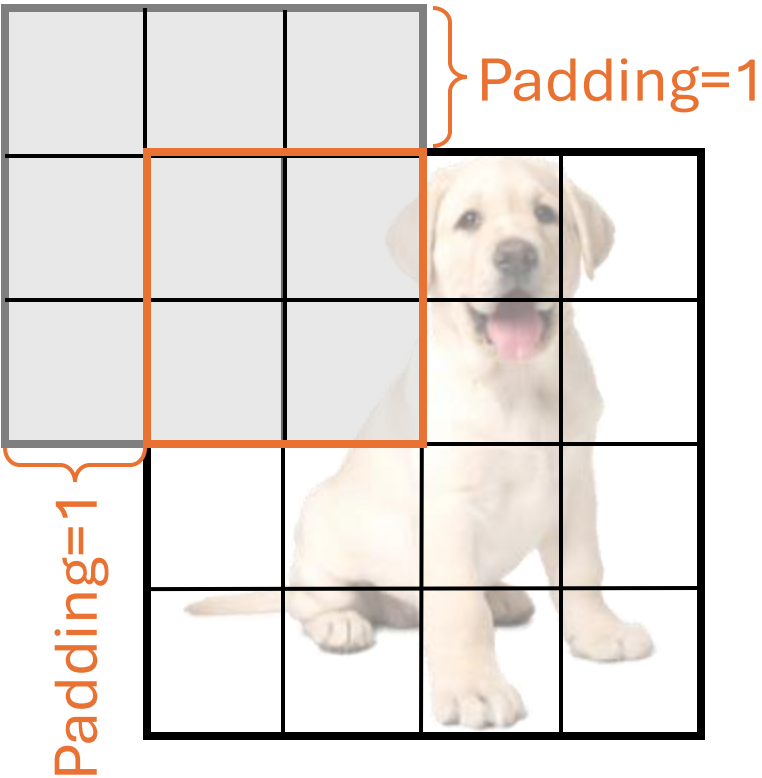
Stride



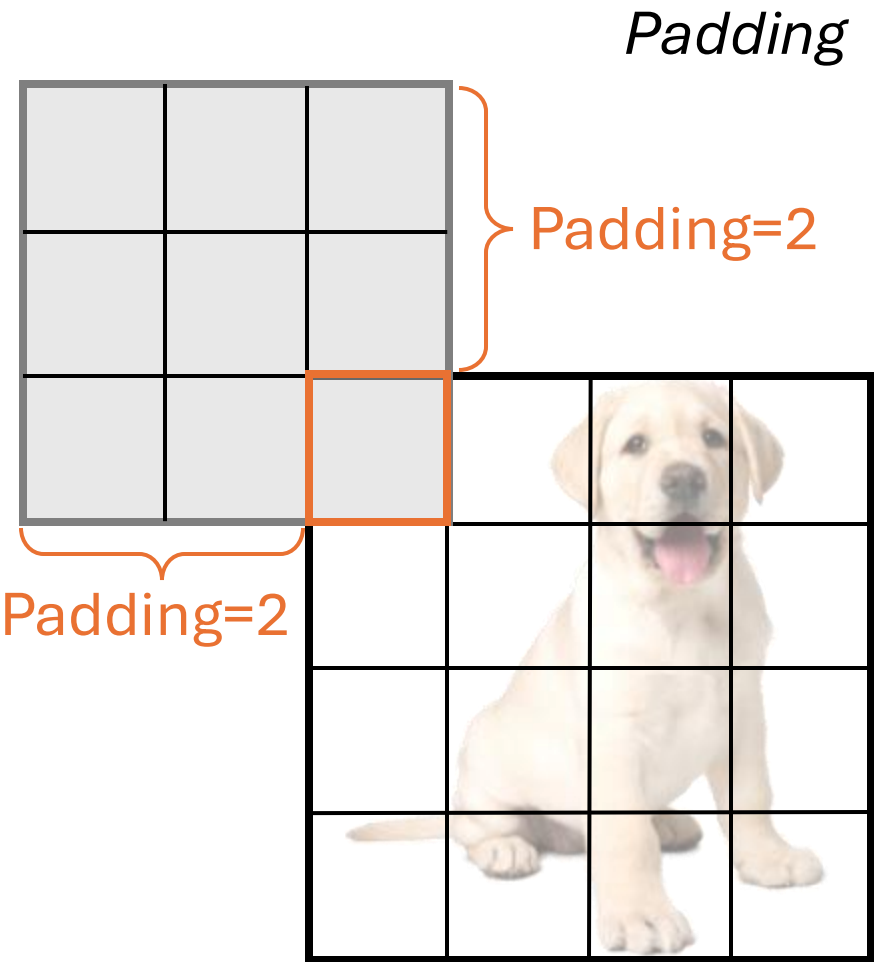
Convolutional Neural Networks (CNN)



No padding (padding=0)
4×4 input → 2×2 output
(shrinks)



Padding=1
4×4 input → 4×4 output
(same size)

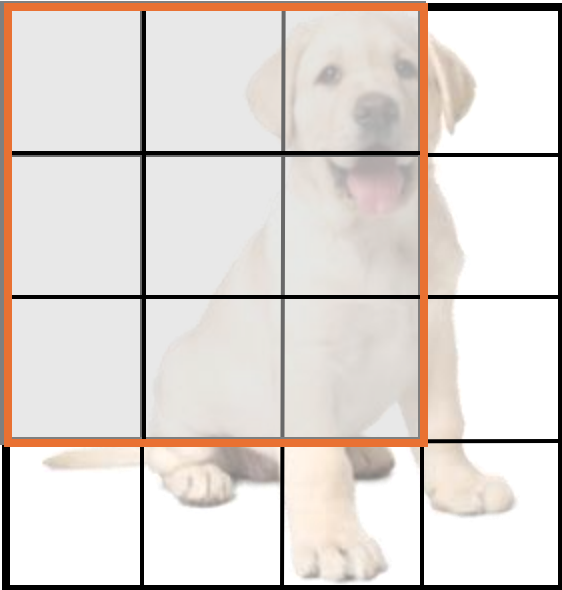


Padding=2
4×4 input → 6×6 output
(grows)

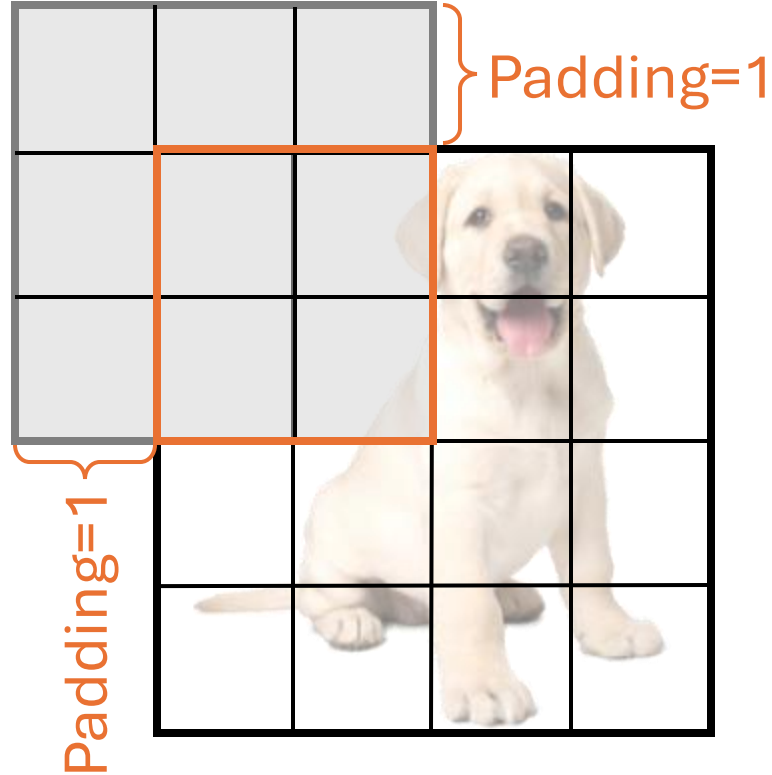


Convolutional Neural Networks (CNN)

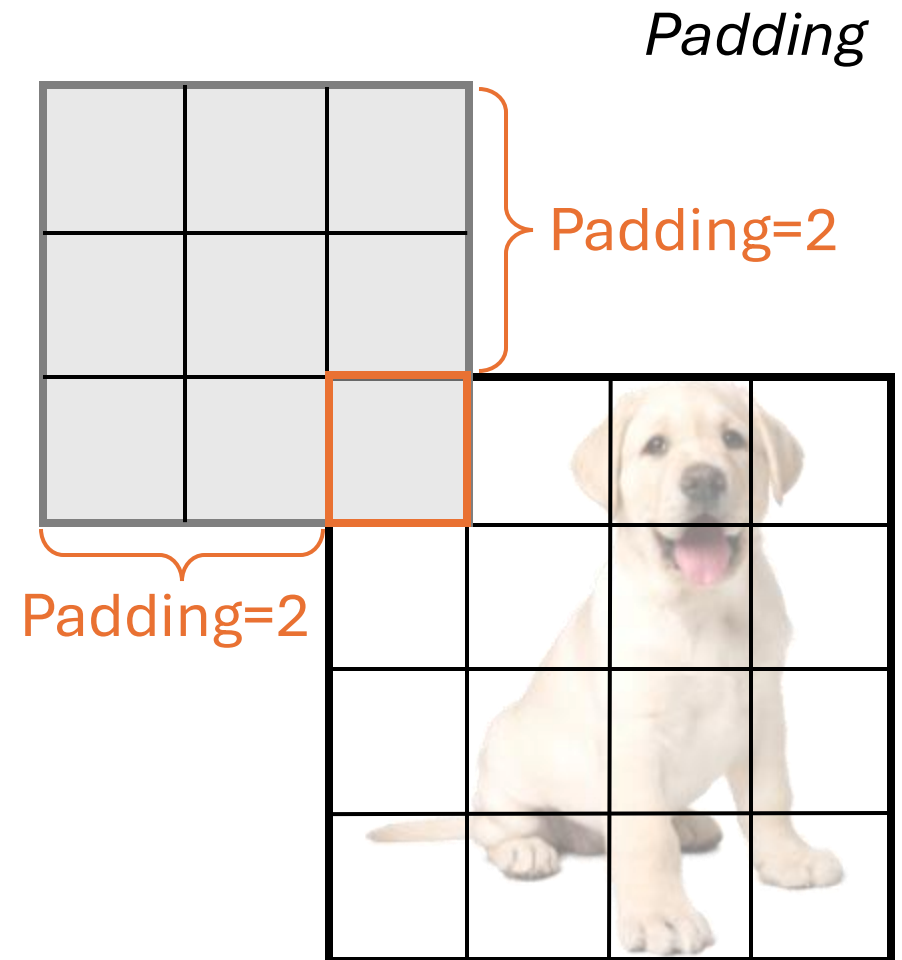
Padding=0



No padding (padding=0)
4×4 input → 2×2 output
(shrinks)



Padding=1
4×4 input → 4×4 output
(same size)



Padding=2
4×4 input → 6×6 output
(grows)

To preserve input size: $\text{padding} = (\text{filter_size} - 1) // 2$
 $\text{output} = (\text{input} + 2 * \text{padding} - \text{filter_size}) / \text{stride} + 1$

Convolutional Neural Networks (CNN)

Convolution Filters (kernels)

```
tf.keras.layers.Conv2D(  
    filters=n,  
    kernel_size=(h_f, w_f),  
    stride=s, padding=p,  
    input_shape=(h_in, w_in, c_in))
```



```
torch.nn.Conv2d(  
    kernel_size=(h_f, w_f),  
    stride=s, padding=p,  
    in_channels=c_in, out_channels=n,  
    input_shape=(c_in, h_in, w_in))
```



i_{11}	i_{12}	i_{13}	i_{14}
i_{21}	i_{22}	i_{23}	i_{24}
i_{31}	i_{32}	i_{33}	i_{34}
i_{41}	i_{42}	i_{43}	i_{44}

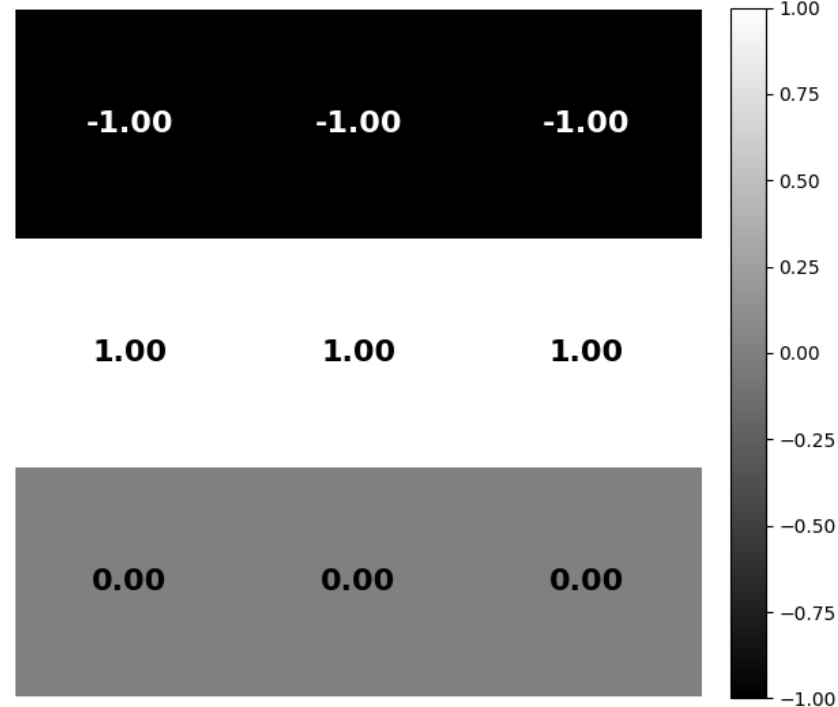
Input tensor

f_{11}	f_{12}	f_{13}
f_{21}	f_{22}	f_{23}
f_{31}	f_{32}	f_{33}

Filter (kernel) tensor

Convolutional Neural Networks (CNN)

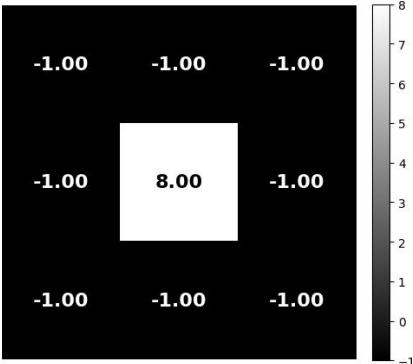
Some conventional filters



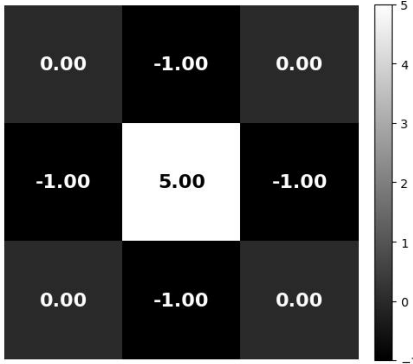
Convolutional Neural Networks (CNN)

Some conventional filters

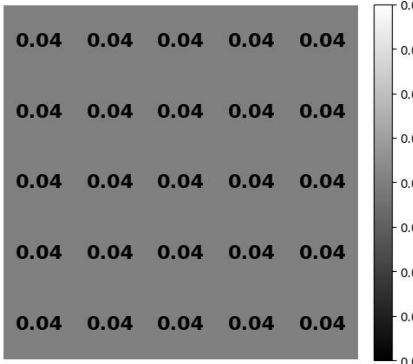
Edge Detection (Laplacian)



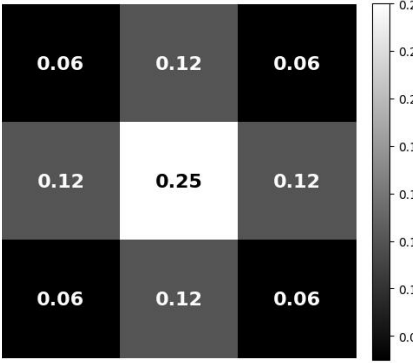
Sharpen Filter



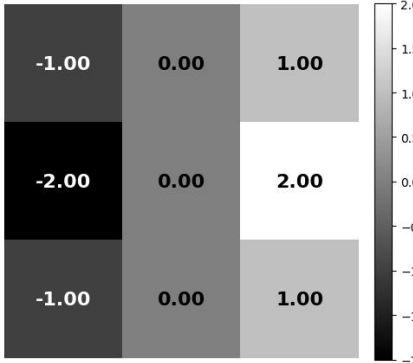
Box Blur (5x5)



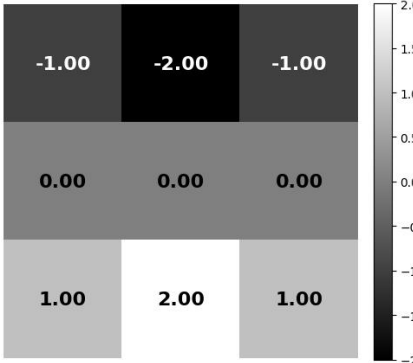
Gaussian Blur (3x3)



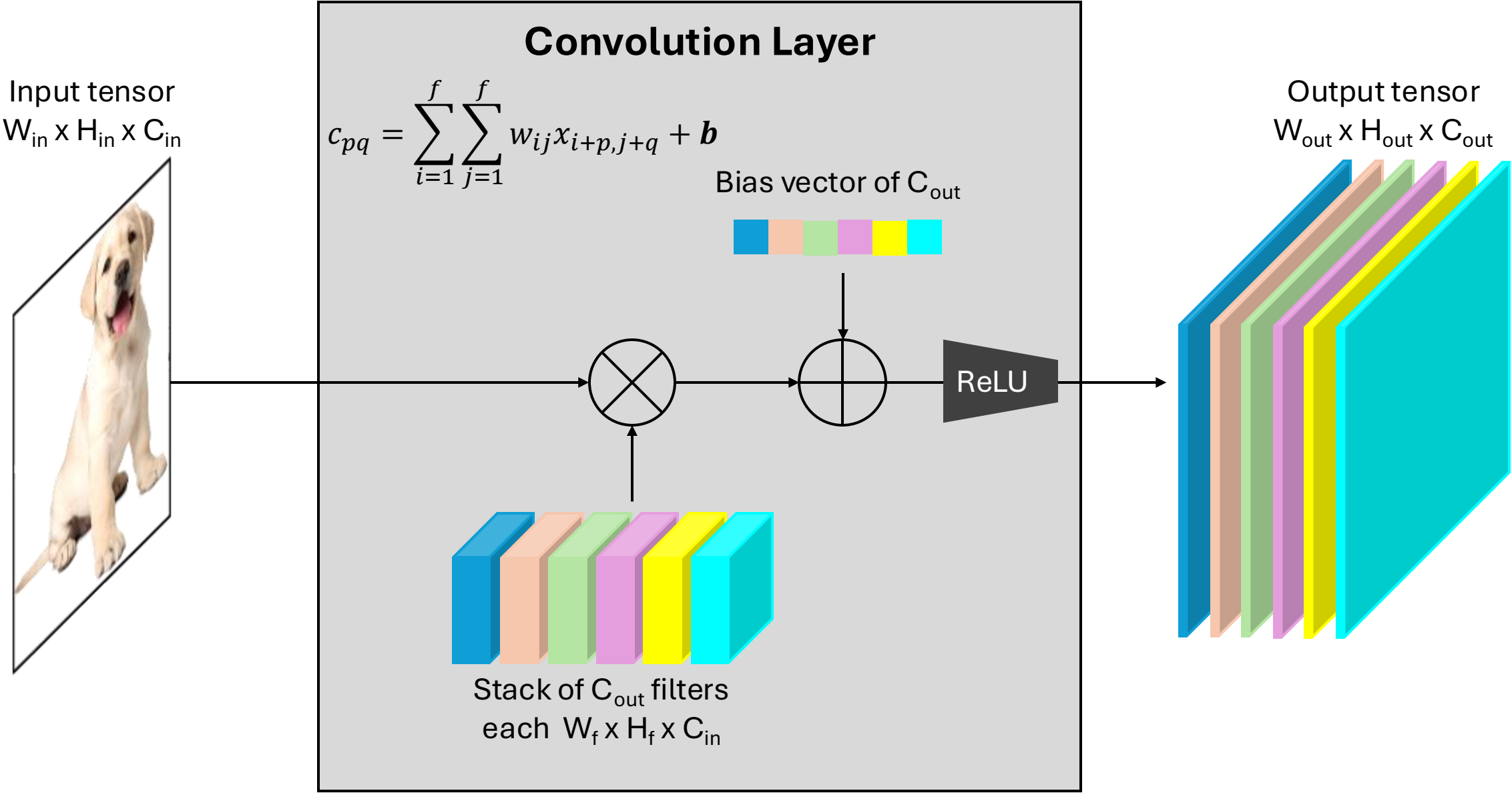
Sobel Edge Detection (X-direction)



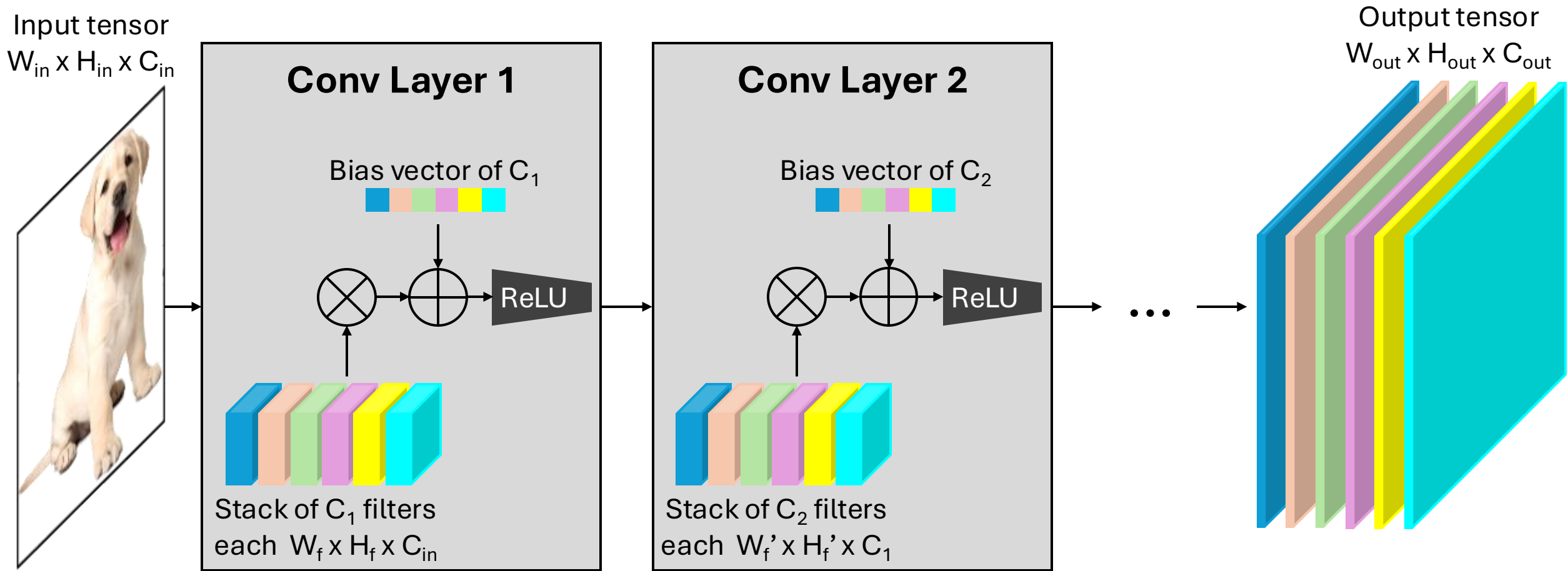
Sobel Edge Detection (Y-direction)



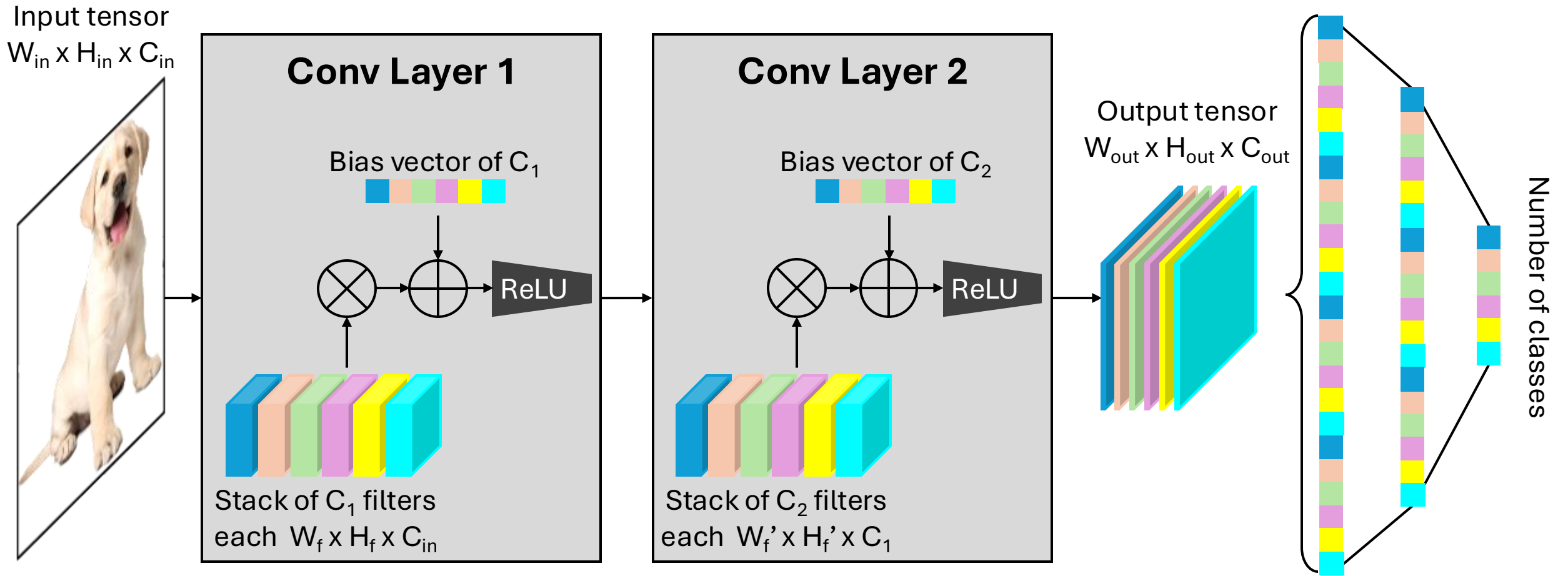
Convolutional Neural Networks (CNN)



Convolutional Neural Networks (CNN)



Convolutional Neural Networks (CNN)



Convolutional Neural Networks (CNN)

Pooling

- ✓ Dimension Reduction
- ✓ Spatial invariance

7	4	8	5
7	3	7	8
5	4	8	8
3	6	5	2

Max Pooling
2x2, stride=2

7	8
6	8

Avg Pooling
2x2, stride=2

5.25	7.0
4.5	5.75

```
import tensorflow as tf  
  
tf.keras.layers.MaxPooling2D(  
    pool_size=(h_f,w_f),  
    strides=s)
```

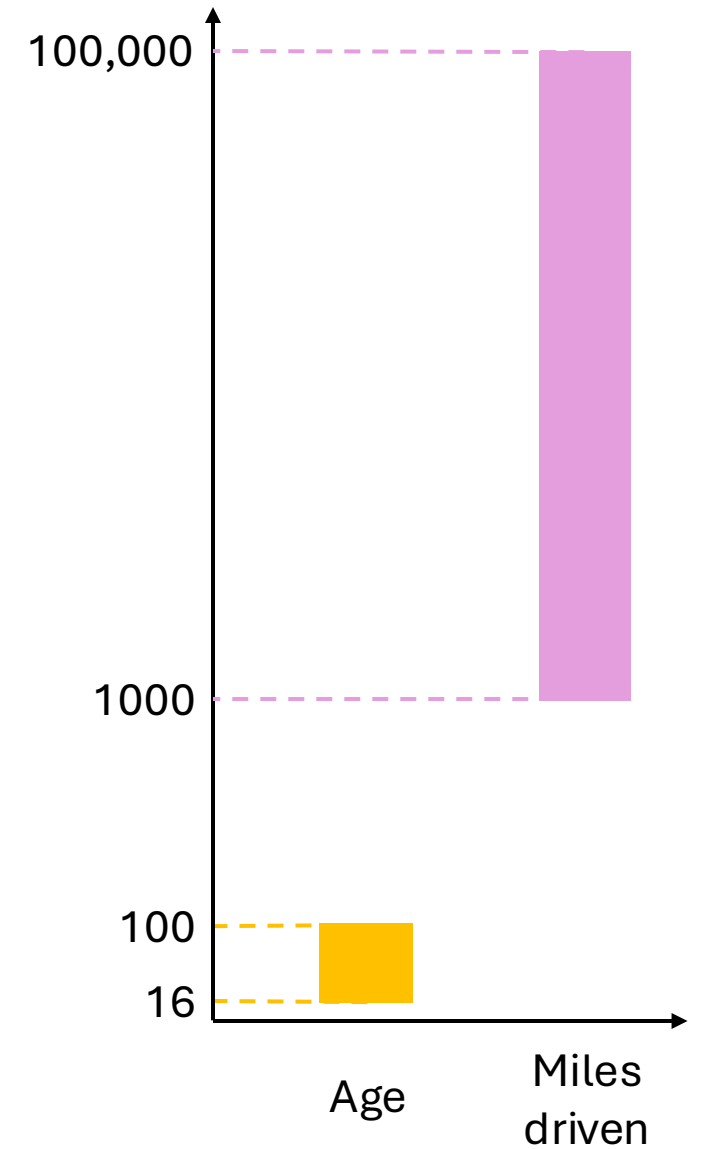


```
import torch  
  
torch.nn.MaxPool2d(  
    kernel_size=(h_f,w_f),  
    stride=s)
```



Convolutional Neural Networks (CNN)

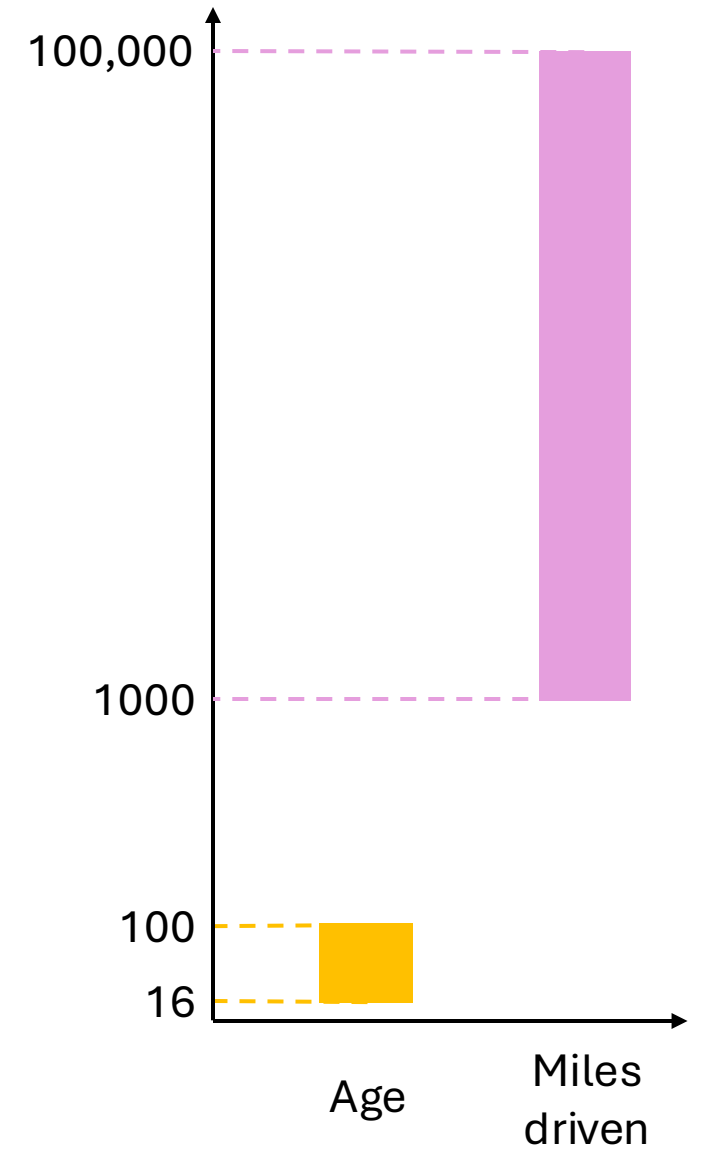
Batch normalization & Dropout



Convolutional Neural Networks (CNN)

- Biased outputs

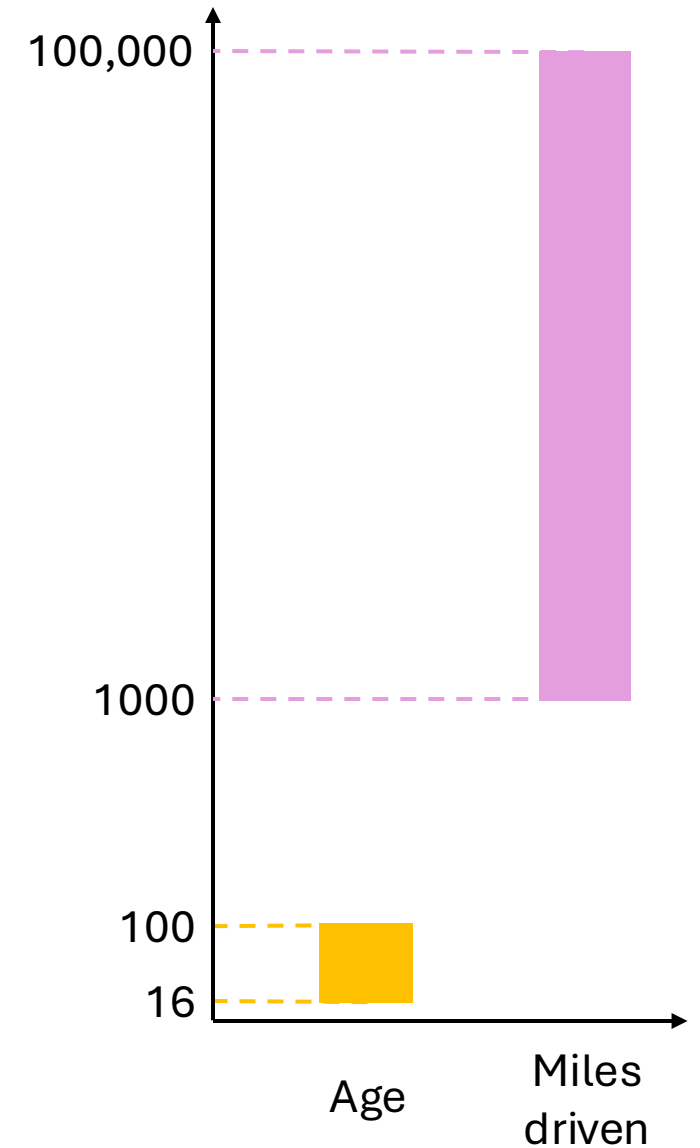
Batch normalization & Dropout



Convolutional Neural Networks (CNN)

- Biased outputs
- Inefficient training

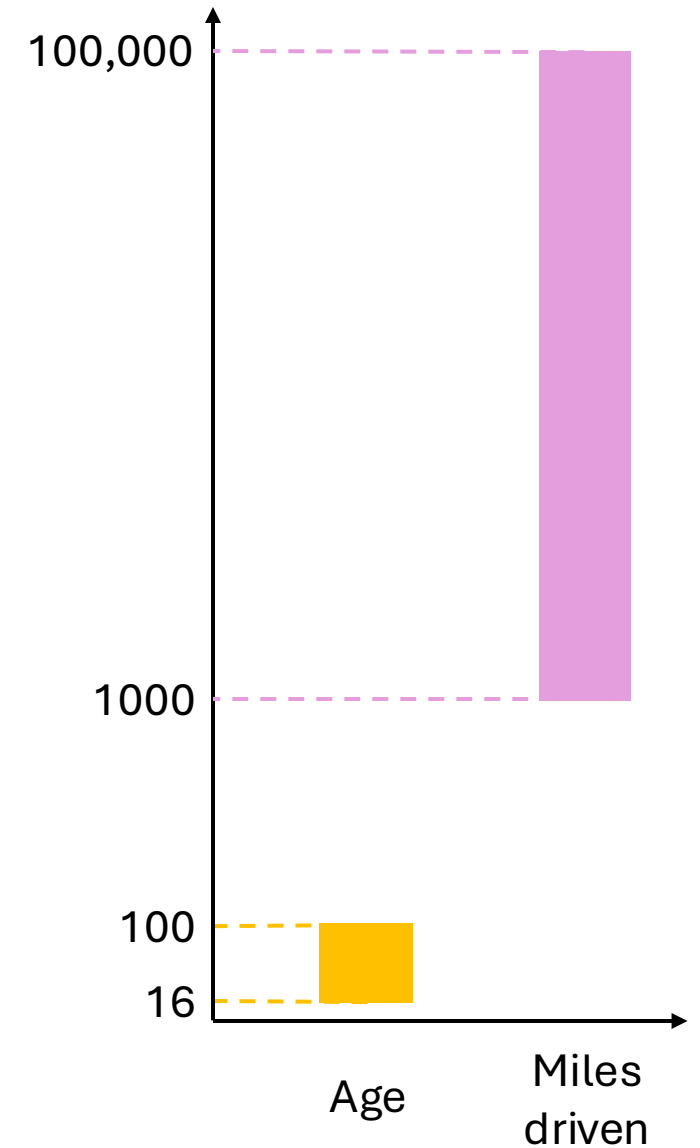
Batch normalization & Dropout



Convolutional Neural Networks (CNN)

- Biased outputs
- Inefficient training
- Gradient explosion

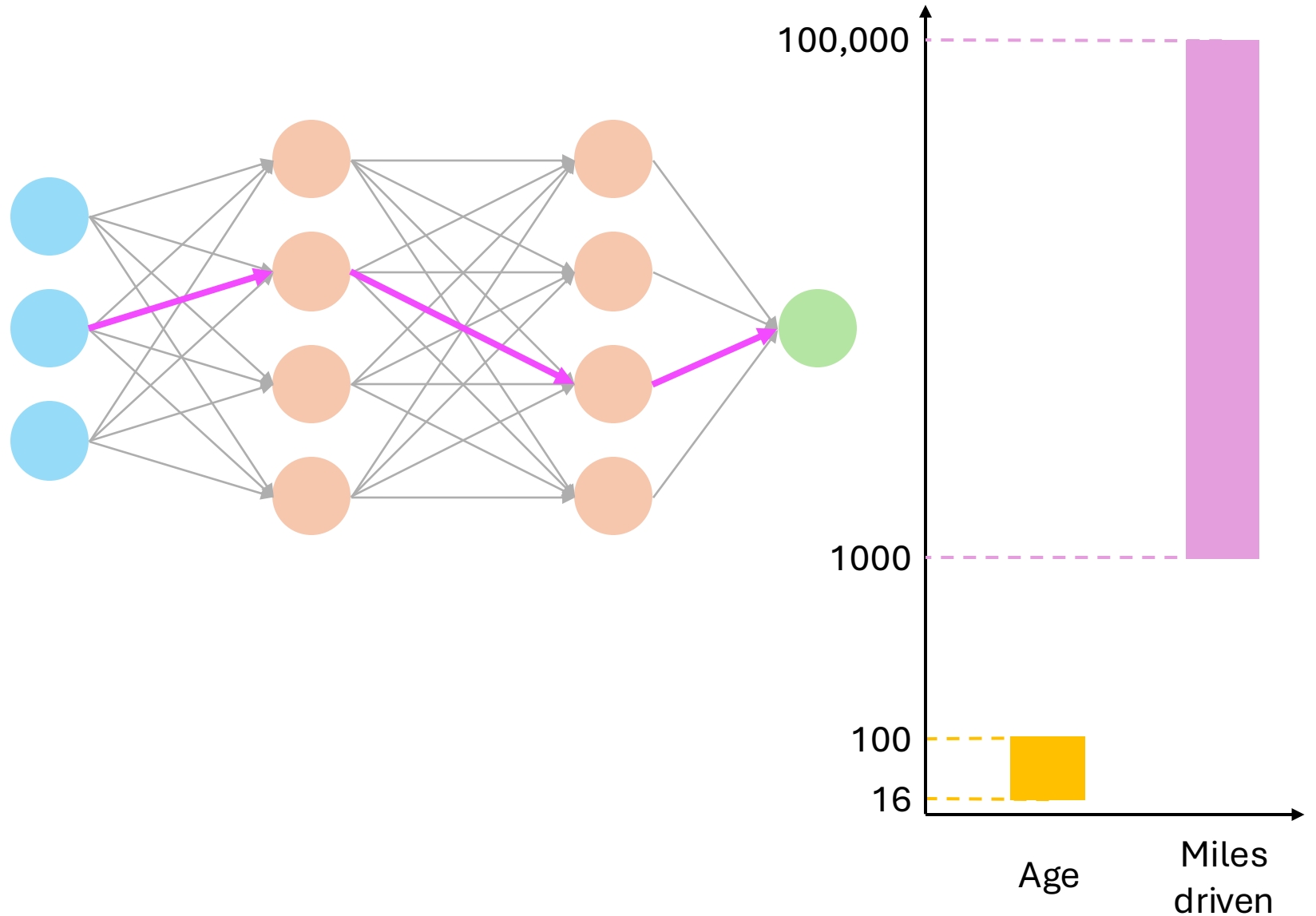
Batch normalization & Dropout



Convolutional Neural Networks (CNN)

Batch normalization & Dropout

- Biased outputs
- Inefficient training
- Gradient explosion
- Imbalanced weights



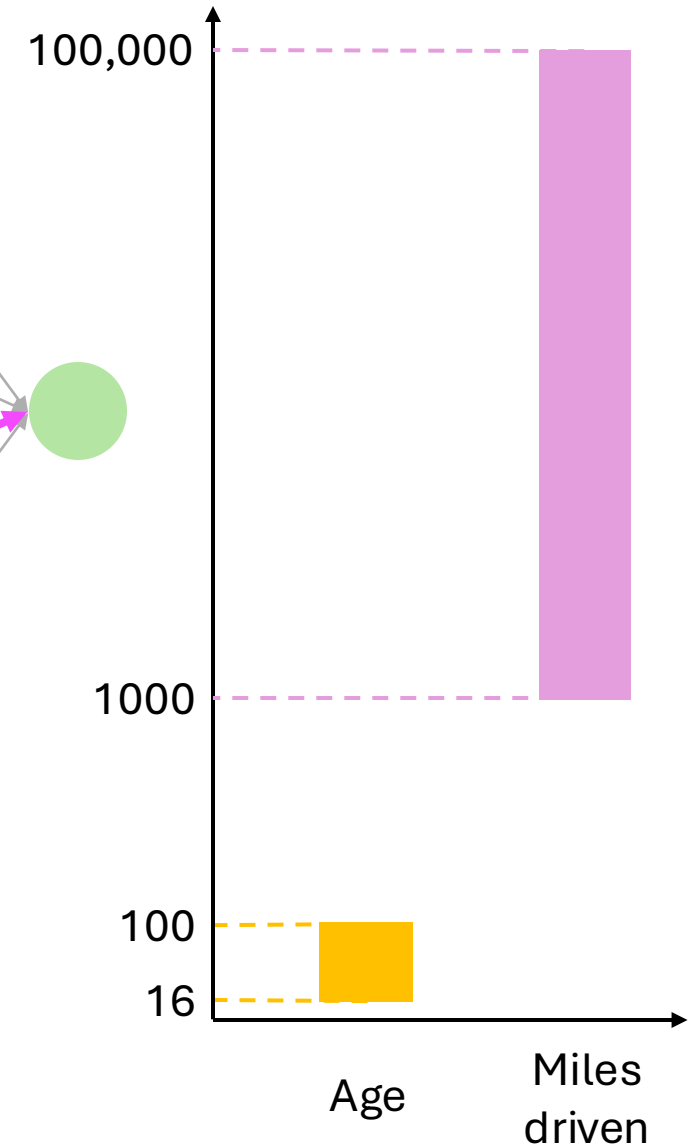
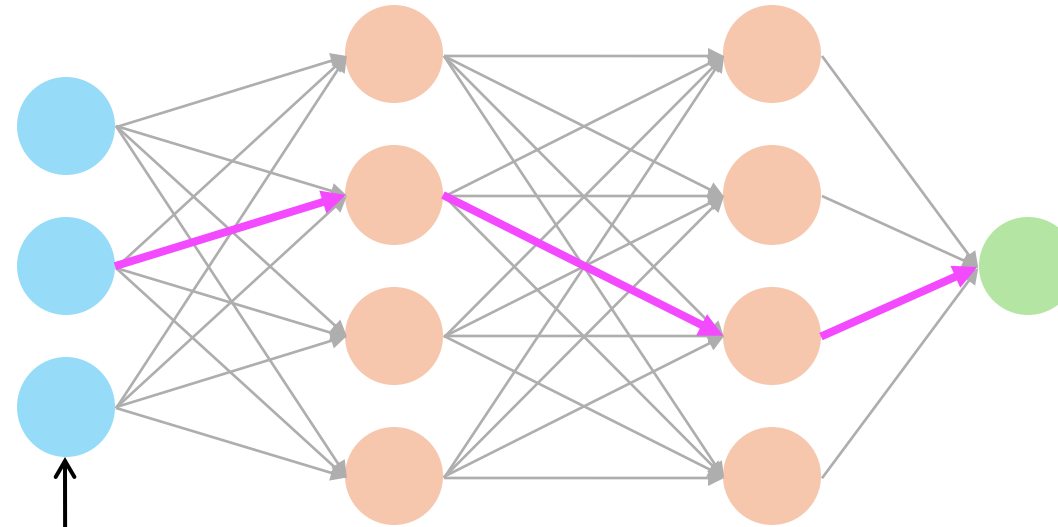
Convolutional Neural Networks (CNN)

Batch normalization & Dropout

- Biased outputs
- Inefficient training
- Gradient explosion
- Imbalanced weights

$$z = \frac{x - \mu}{\sigma}$$

Input/feature norm.

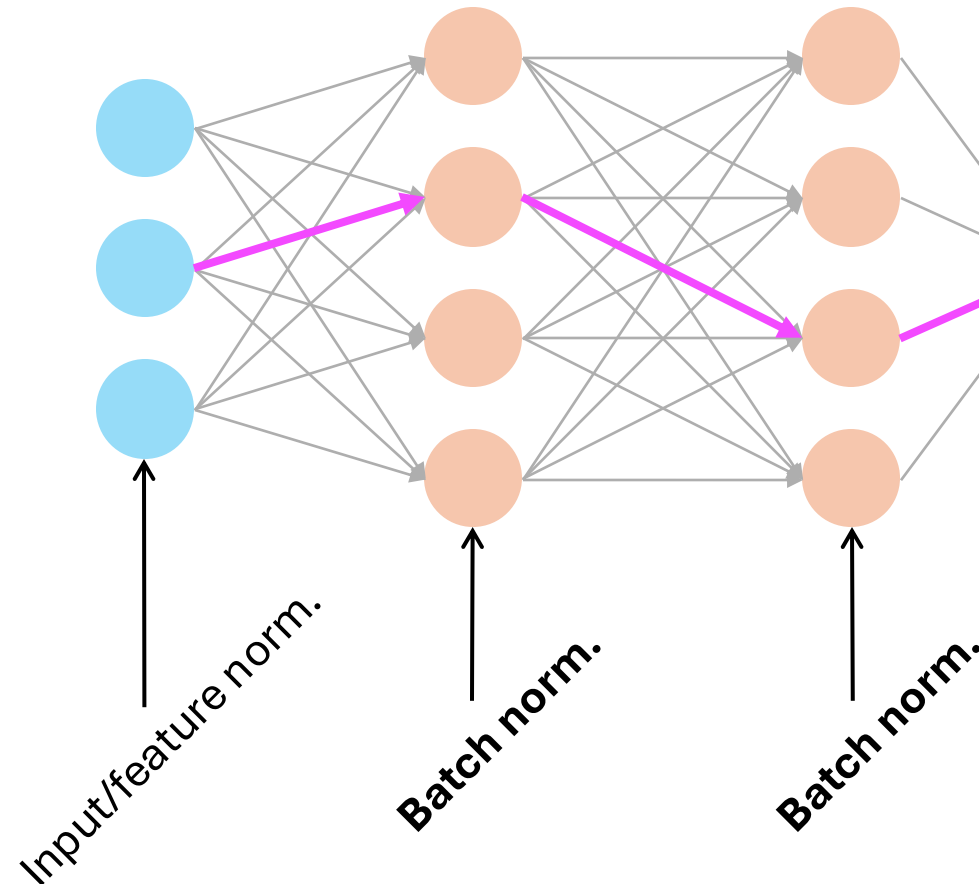


Convolutional Neural Networks (CNN)

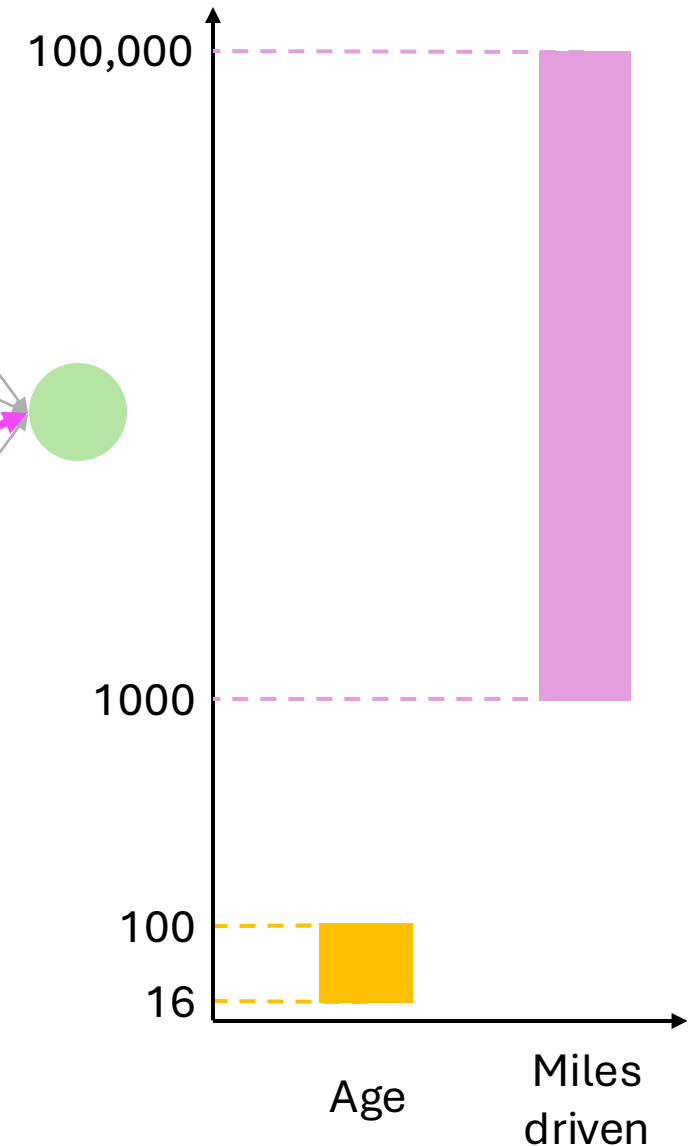
Batch normalization & Dropout

- Biased outputs
- Inefficient training
- Gradient explosion
- Imbalanced weights

$$z = \frac{x - \mu}{\sigma}$$



$$z_{l+1} = g \times z_l + b$$

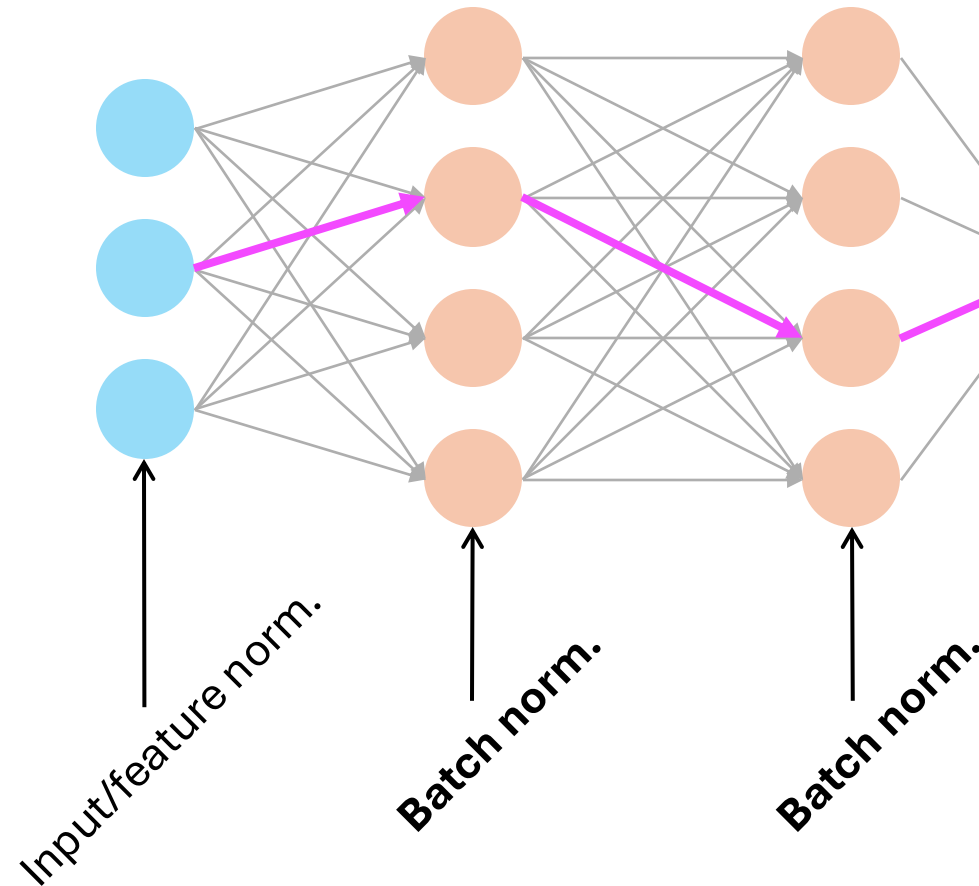


Convolutional Neural Networks (CNN)

Batch normalization & Dropout

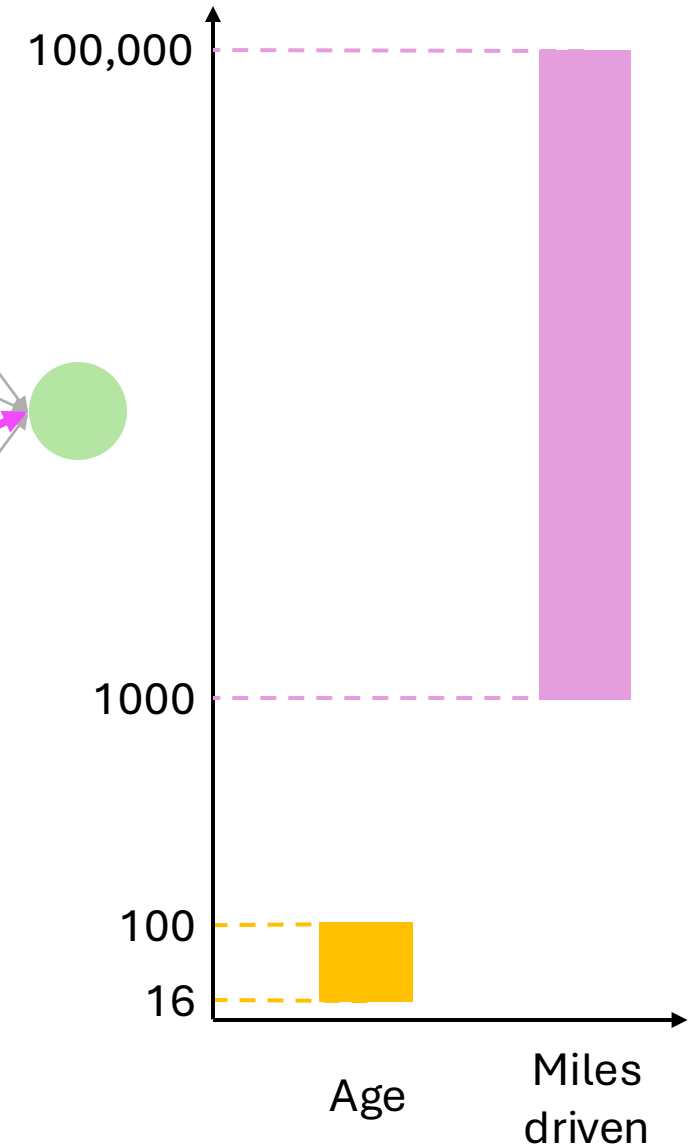
- Biased outputs
- Inefficient training
- Gradient explosion
- Imbalanced weights

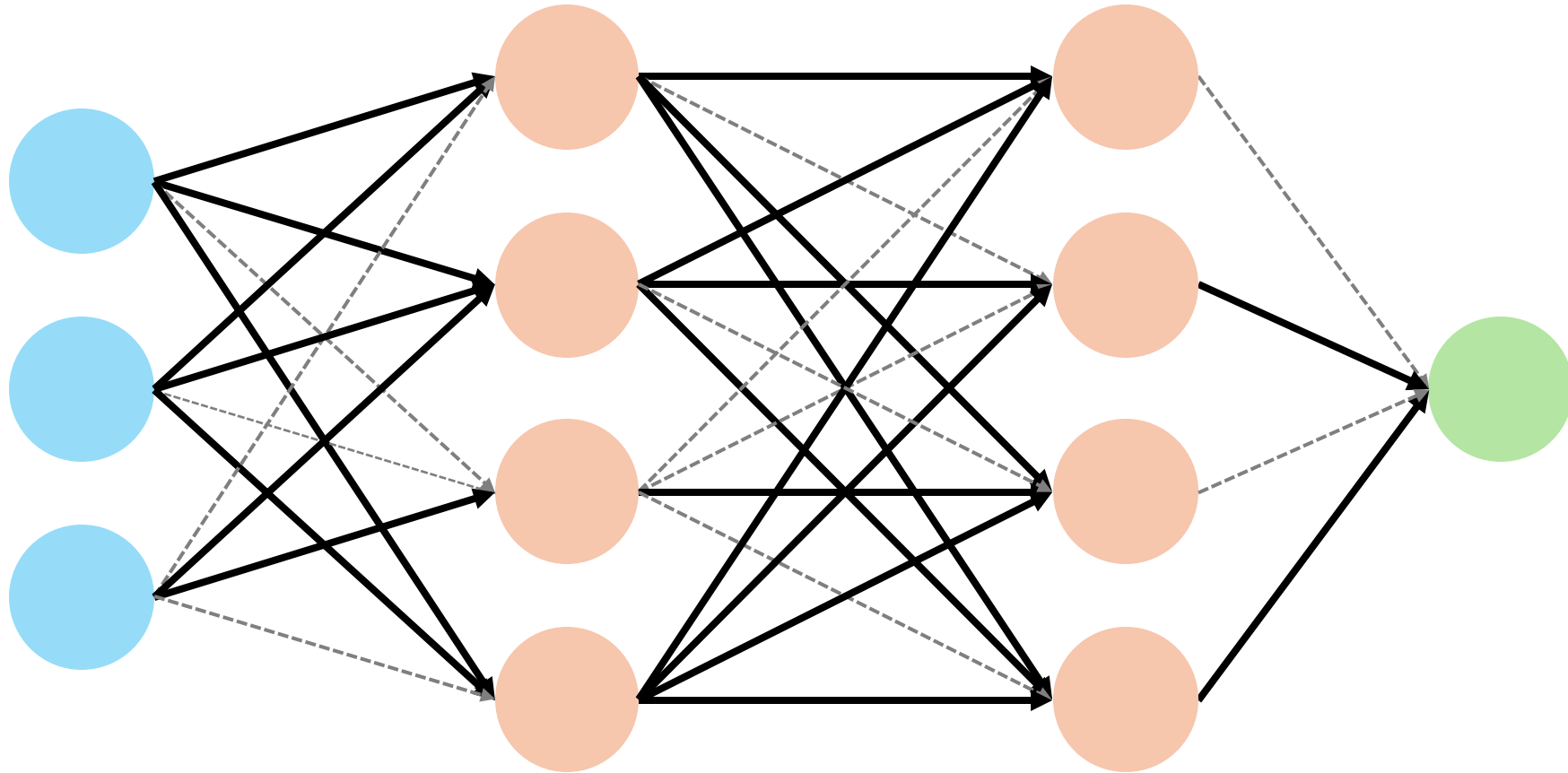
$$z = \frac{x - \mu}{\sigma}$$



$$z_{l+1} = \boxed{g} \times z_l + \boxed{b}$$

trainable







Break



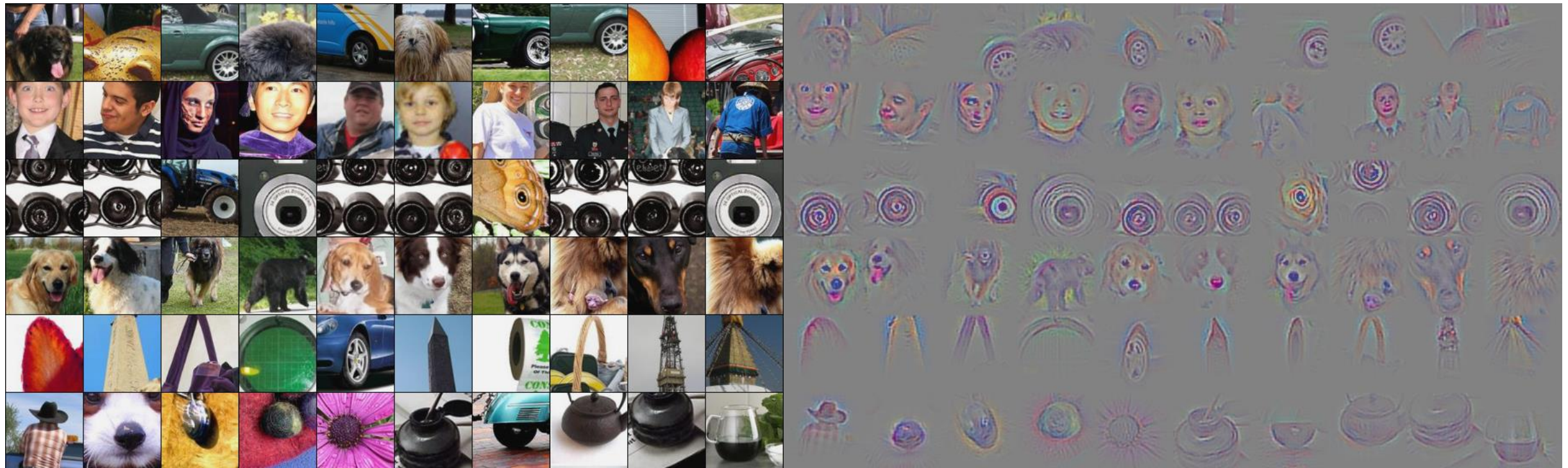
(Q&A)

Convolutional Neural Networks (CNN)

Conv Layer 6



Conv Layer 9



```
import tensorflow as tf

model = tf.keras.Sequential([
    # first convolutional layer
    tf.keras.layers.Conv2D(32, kernel_size=3, activation='relu'),
    tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

    # second convolutional layer
    tf.keras.layers.Conv2D(64, kernel_size=3, activation='relu'),
    tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

    # fully-connected classifier
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1024, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.build(input_shape=(None, 32, 32, 3))
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 1024)	2,360,320
dense_3 (Dense)	(None, 10)	10,250

Total params: 2,389,962 (9.12 MB)

Trainable params: 2,389,962 (9.12 MB)

Non-trainable params: 0 (0.00 B)




```
import torch
```

```
model = torch.nn.Sequential(  
    # first convolutional layer  
    torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3),  
    torch.nn.ReLU(),  
    torch.nn.MaxPool2d(kernel_size=2, stride=2),  
  
    # second convolutional layer  
    torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),  
    torch.nn.ReLU(),  
    torch.nn.MaxPool2d(kernel_size=2, stride=2),  
  
    # fully-connected classifier  
    torch.nn.Flatten(),  
    torch.nn.Linear(in_features=64 * 6 * 6, out_features=1024),  
    torch.nn.ReLU(),  
    torch.nn.Linear(in_features=1024, out_features=10)  
)
```

```
input_tensor = torch.randn(1, 3, 32, 32) # batch size of 1
```

```
output = model(input_tensor)
```

```
print(model)
```

```
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
```

```
Sequential(  
  (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1))  
  (1): ReLU()  
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))  
  (4): ReLU()  
  (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  (6): Flatten(start_dim=1, end_dim=-1)  
  (7): Linear(in_features=2304, out_features=1024, bias=True)  
  (8): ReLU()  
  (9): Linear(in_features=1024, out_features=10, bias=True)  
)  
Total parameters: 2,389,962  
Trainable parameters: 2,389,962
```



Convolutional Neural Networks (CNN)

Quiz time!

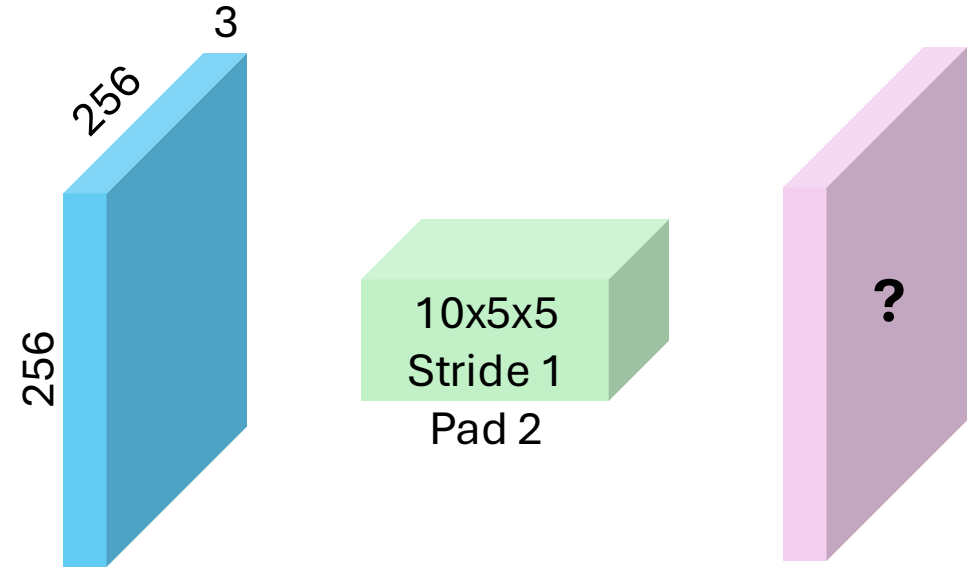
Input tensor: $256 \times 256 \times 3$

Conv kernels: $10 \times 5 \times 5$

Stride 1 and padding 2

What would be the output tensor size?

How many trainable parameters?



Convolutional Neural Networks (CNN)

Quiz time!

Input tensor: 256 x 256 x 3

Conv kernels: 10 x 5 x 5

Stride 1 and padding 2

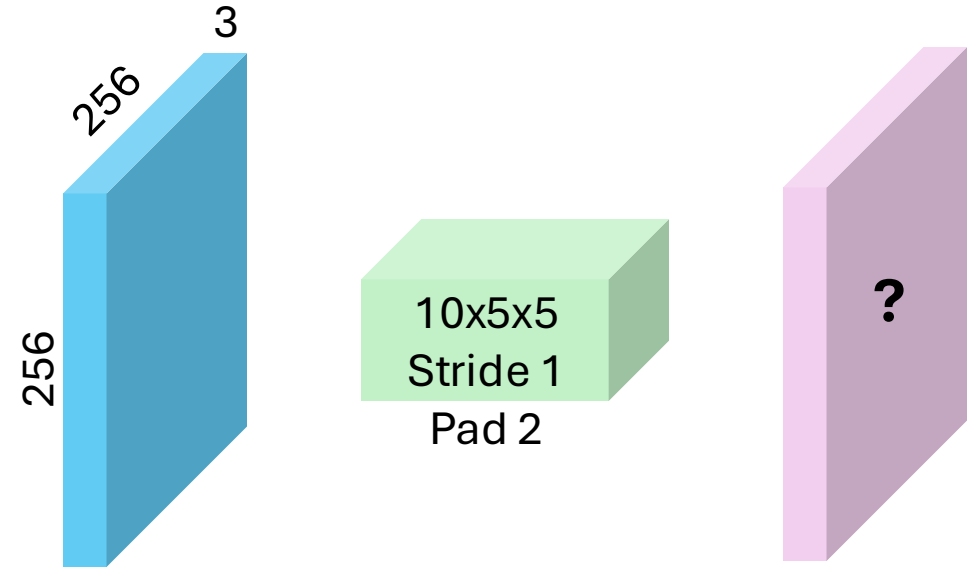
What would be the output tensor size?

$(256 + 2 \times 2 - 5) / 1 + 1 = 256 \Rightarrow 10 \times 256 \times 256$

How many trainable parameters?

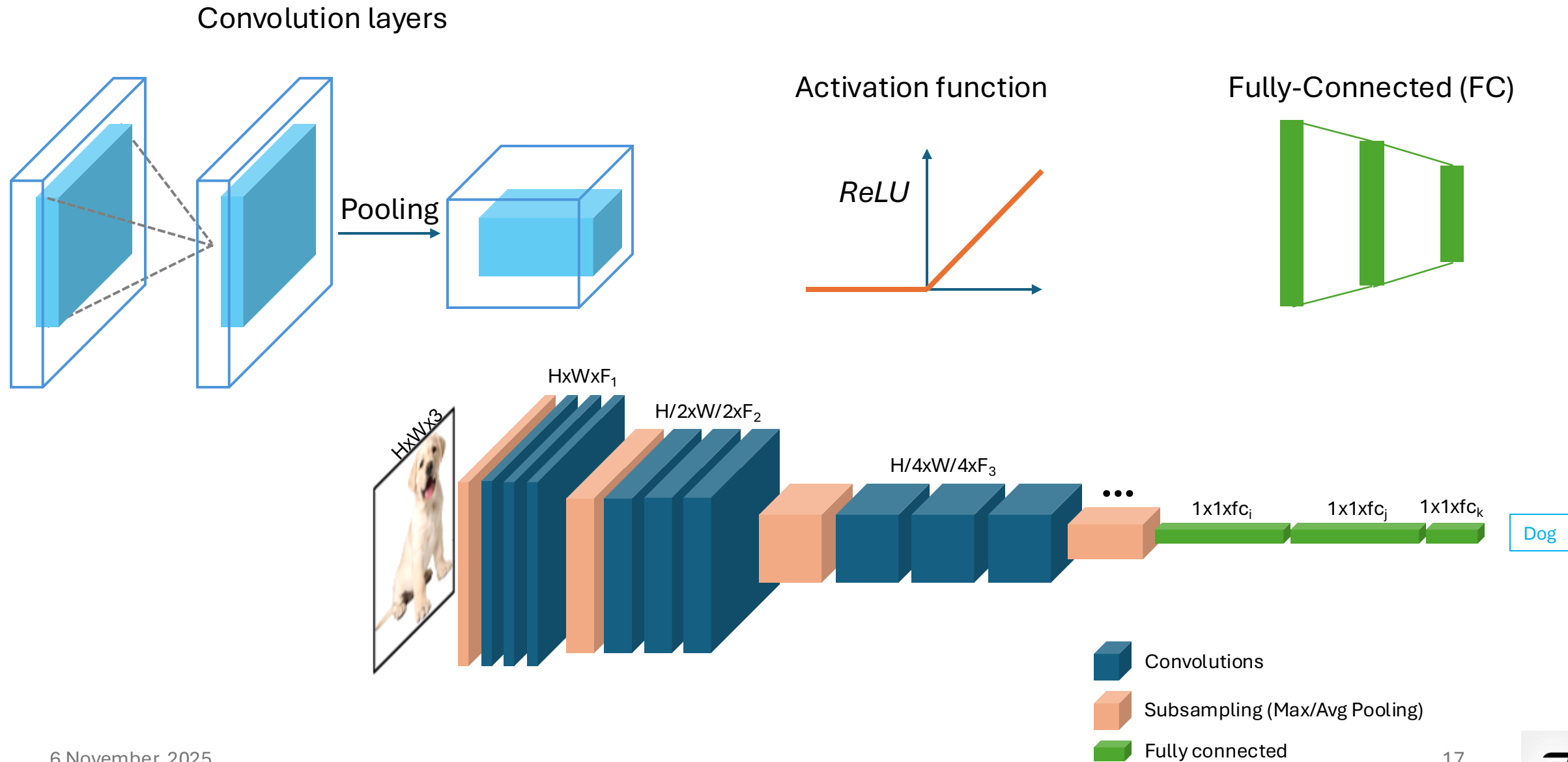
Every filter: $3 \times 5 \times 5 + 1 (\text{bias}) = 76$

$10 \times 76 = 760$ trainable parameters



Convolutional Neural Networks (CNN)

Summary:



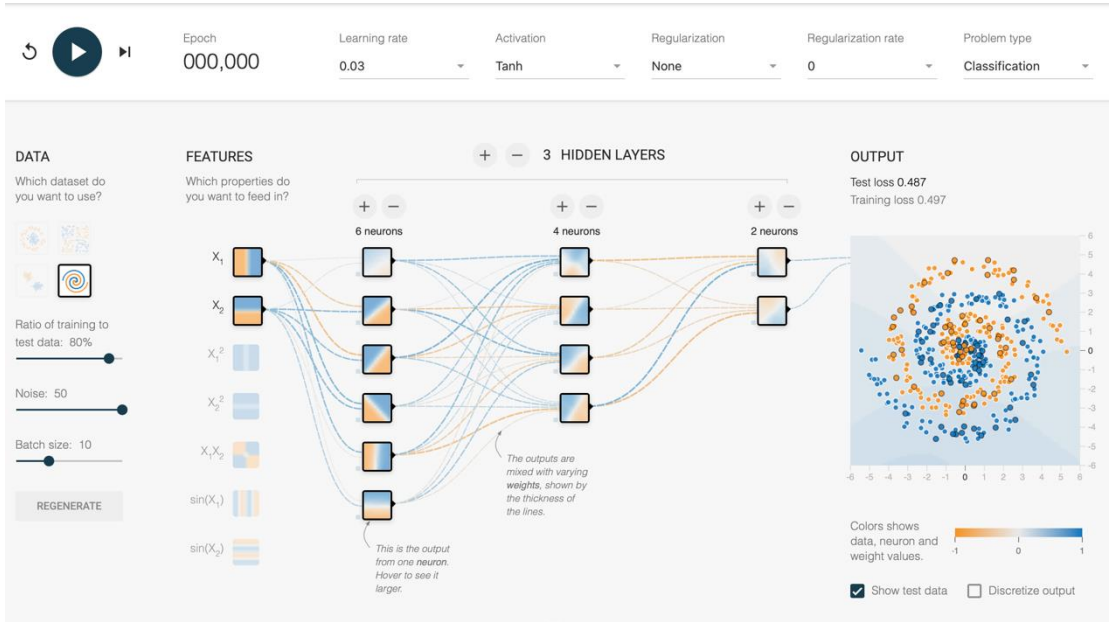
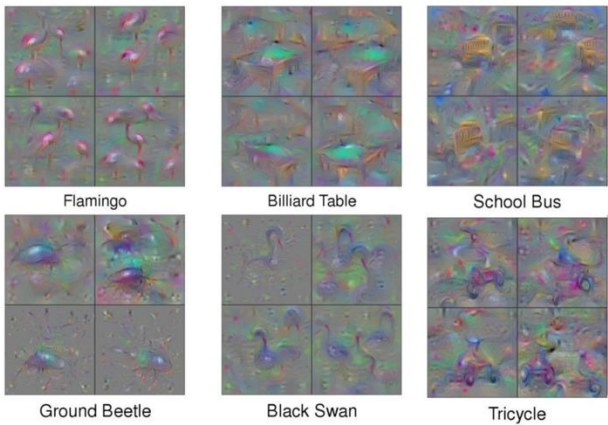
Convolutional Neural Networks (CNN)

Playgrounds:

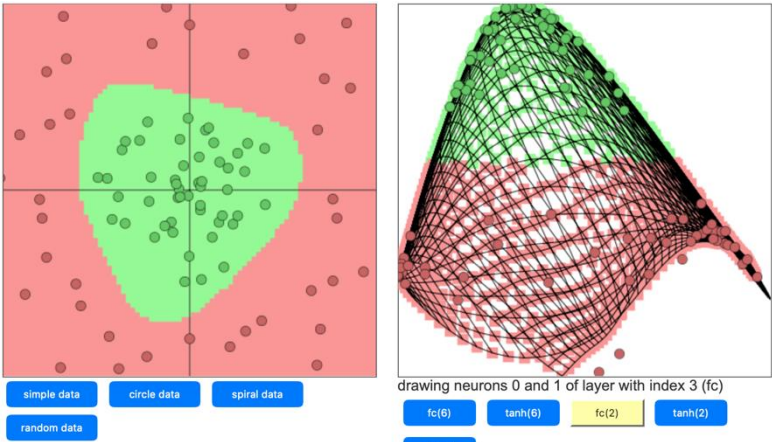
playground.tensorflow.org

Understanding Neural Networks Through Deep Visualization
Yosinski, et al, ICML 2015 DL Workshop

<https://yosinski.com/deepvis>



ConvNetJS



Controls:
CLICK: Add red data point
SHIFT+CLICK: Add green data point
CTRL+CLICK: Remove closest data point



(Q&A)

Parham Kebria

 parhamkebria@ieee.org